

Introducción a los algoritmos avanzados y complejidad computacional

Algoritmos Avanzados – Sesión teórica 1

Docente: Héctor Eduardo Ugarte Rojas

Universidad Nacional de San Antonio Abad del Cusco
Carrera Profesional de Ingeniería Informática y de Sistemas

Semestre académico 2026-II

Propósito y resultados esperados

Propósito: construir un marco para reconocer problemas computacionales complejos y seleccionar estrategias algorítmicas pertinentes.

Al finalizar la sesión, el estudiante podrá:

- 1 distinguir problemas de decisión, búsqueda y optimización;
- 2 interpretar complejidades polinomiales, pseudo-polinomiales y exponenciales;
- 3 diferenciar P, NP, NP-hard y NP-completo;
- 4 relacionar fuerza bruta, Programación Dinámica, Ramificar-Podar, aproximación y heurísticas;
- 5 justificar inicialmente la elección de una estrategia algorítmica.

La sesión dentro de la asignatura

Conocimientos que se recuperan

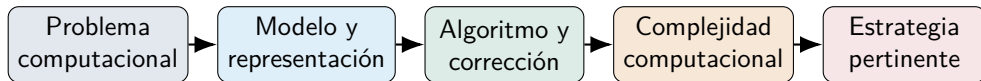
- análisis asintótico;
- Programación Dinámica;
- Ramificar-Podar;
- algoritmos de grafos;
- estructuras de datos.

Nuevas conexiones

- complejidad computacional;
- aproximación;
- análisis amortizado;
- algoritmos probabilísticos;
- aplicaciones algebraicas y criptográficas.

Enfoque

Programación Dinámica y Ramificar-Podar no se estudiarán nuevamente desde cero: se utilizarán como herramientas para comparar y construir soluciones avanzadas.



Pregunta guía

¿Debemos buscar siempre la solución óptima, aunque el tiempo requerido resulte prohibitivo?

¿Qué entendemos por algoritmo avanzado?

No existe una frontera única. En esta asignatura, un algoritmo se considera avanzado cuando exige una o más de estas capacidades:

- modelar un problema combinatorio o algebraico no trivial;
- aprovechar estructura para evitar una búsqueda exhaustiva;
- utilizar estructuras de datos especializadas;
- demostrar corrección o una garantía de aproximación;
- analizar costos amortizados, probabilísticos o parametrizados;
- contrastar eficiencia teórica con desempeño experimental.

Idea central

“Avanzado” no significa únicamente código complicado, sino decisiones de diseño sustentadas por propiedades y análisis.

Problema, instancia y algoritmo

Problema

Familia general de entradas y una especificación de la salida requerida.

Instancia

Caso concreto del problema, con datos específicos.

Algoritmo

Procedimiento finito y preciso que transforma cada entrada válida en una salida que satisface la especificación.

Ejemplo

Problema: encontrar un camino mínimo. Instancia: un grafo ponderado, un origen y un destino determinados.

Tres formas de plantear un problema

Decisión

Respuesta sí/no.

¿Existe una asignación con makespan a lo sumo K ?

Búsqueda

Se solicita un objeto válido.

Encuentre una asignación con makespan a lo sumo K .

Optimización

Se solicita el mejor objeto válido.

Encuentre una asignación de makespan mínimo.

Observación

Las clases P y NP se formulan normalmente sobre problemas de decisión, aunque las aplicaciones sean de búsqueda u optimización.

Correctitud, complejidad y pertinencia

Correctitud: ¿el algoritmo produce la salida requerida para toda entrada válida?

Complejidad: ¿cómo crecen el tiempo y el espacio cuando aumenta la entrada?

Pertinencia: ¿la solución satisface las restricciones reales de tamaño, tiempo, memoria y calidad?

Un algoritmo correcto puede no ser pertinente

Enumerar todas las asignaciones puede garantizar el óptimo, pero resultar inviable cuando el número de trabajos crece.

Regla de trabajo

Primero se valida la corrección; después se interpretan el rendimiento y la calidad de la solución.

Entrada, tamaño y modelo de costo

Sea x una instancia y $|x|$ la longitud de su representación.

- El tiempo $T(n)$ se expresa en función de $n = |x|$.
- El valor numérico de un dato no es necesariamente su tamaño.
- Un entero positivo N requiere $\lfloor \log_2 N \rfloor + 1$ bits.
- El costo de las operaciones depende del modelo de cómputo asumido.

Ejemplo

Un algoritmo que ejecuta N iteraciones no es polinomial en la longitud binaria de N :

$$N = 2^{\log_2 N}.$$

Órdenes de crecimiento

Crecimiento	Ejemplo	Interpretación inicial
Constante	$\mathcal{O}(1)$	No crece con n bajo el modelo asumido.
Logarítmico	$\mathcal{O}(\log n)$	Reduce repetidamente el espacio de búsqueda.
Lineal	$\mathcal{O}(n)$	Procesa una cantidad proporcional a la entrada.
Cuasilineal	$\mathcal{O}(n \log n)$	Frecuente en ordenamiento y FFT.
Polinomial	$\mathcal{O}(n^k)$	Tratable en el sentido teórico, con matices.
Exponencial	$\mathcal{O}(c^n)$	Se deteriora rápidamente al crecer n .
Factorial	$\mathcal{O}(n!)$	Frecuente al enumerar permutaciones.

¿Qué ocurre cuando crece la entrada?

Si n se duplica, de manera orientativa:

Costo	Factor aproximado
$\mathcal{O}(n)$	2
$\mathcal{O}(n \log n)$	algo mayor que 2
$\mathcal{O}(n^2)$	4
$\mathcal{O}(n^3)$	8

Para $\mathcal{O}(2^n)$, aumentar n en una unidad duplica el espacio de posibilidades:

$$2^{n+1} = 2 \cdot 2^n.$$

Matiz

Los factores son tendencias asintóticas; las mediciones reales incluyen constantes, memoria y características de las instancias.

Polinomial, pseudo-polinomial y exponencial

Tiempo polinomial

$T(n) = \mathcal{O}(n^k)$ respecto de la longitud total de la entrada.

Tiempo pseudo-polinomial

Es polinomial respecto del valor numérico de un parámetro, pero puede ser exponencial respecto de la cantidad de bits que lo representa.

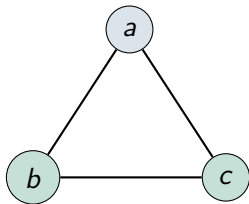
Knapsack mediante Programación Dinámica

Una formulación clásica requiere $\mathcal{O}(nW)$. Como representar W requiere $\Theta(\log W)$ bits, el costo es pseudo-polinomial.

Vertex Cover como problema de decisión

Instancia: un grafo $G = (V, E)$ y un entero k .

Pregunta: ¿existe $C \subseteq V$, con $|C| \leq k$, tal que cada arista tenga al menos un extremo en C ?



En un triángulo, un vértice no basta; cualesquiera dos vértices forman una cobertura.

Definition

P es la clase de problemas de decisión que pueden resolverse en tiempo polinomial mediante un algoritmo determinista.

- Es una noción formal y robusta de eficiencia.
- No significa que todo algoritmo polinomial sea rápido en la práctica.
- Un costo $\mathcal{O}(n^{100})$ es polinomial, pero difícilmente práctico.
- Un algoritmo exponencial puede ser útil sobre entradas pequeñas o estructuradas.

Ejemplos típicos

Camino mínimo con pesos no negativos, árbol de expansión mínima, ordenamiento y máximo común divisor.

Clase NP: certificados verificables

Definition

NP es la clase de problemas de decisión cuyas instancias afirmativas poseen un certificado de longitud polinomial verificable en tiempo polinomial.

Certificado para Vertex Cover

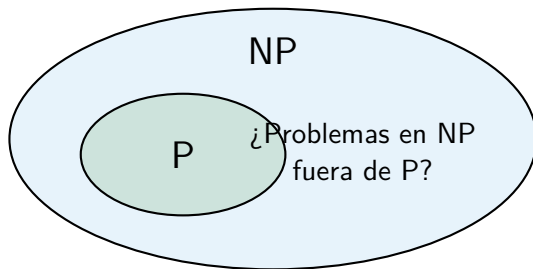
El certificado es el conjunto C . El verificador comprueba:

- 1 que $|C| \leq k$;
- 2 que para cada $(u, v) \in E$, se cumple $u \in C$ o $v \in C$.

No confundir

NP no significa “no polinomial”. Sabemos que $P \subseteq NP$.

P versus NP



$$P \stackrel{?}{=} NP$$

No se conoce si todo problema con certificados verificables eficientemente también puede resolverse eficientemente.

NP-hard y NP-completo

NP-hard

Un problema H es NP-hard si todo problema de NP puede reducirse a H en tiempo polinomial. No necesita pertenecer a NP ni ser un problema de decisión.

NP-completo

Un problema C es NP-completo si:

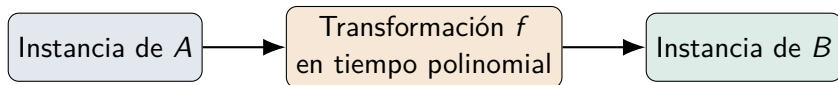
- 1 $C \in \text{NP}$;
- 2 C es NP-hard.

Consecuencia

Un algoritmo polinomial para cualquier problema NP-completo implicaría $P = \text{NP}$.

Reducciones polinomiales

Decimos que $A \leq_p B$ si cualquier instancia de A puede transformarse en tiempo polinomial en una instancia de B , preservando la respuesta.



Dirección para demostrar dificultad

Para demostrar que el problema nuevo B es difícil, se reduce un problema conocido A hacia B : $A \leq_p B$.

¿Qué hacemos ante un problema difícil?

Estrategia	Resultado	Cuándo puede convenir
Fuerza bruta	Óptimo	Instancias pequeñas y validación.
Programación Dinámica	Óptimo en variantes aplicables	Estados o parámetros controlables.
Ramificar-Podar	Óptimo si termina	Buenas cotas y poda efectiva.
Aproximación	Garantía respecto del óptimo	Gran escala y garantía necesaria.
Probabilístico	Error o tiempo controlado	Existe una ganancia sustancial.
Heurística	Sin garantía general	Respuesta rápida y evaluación empírica.
Parametrizado	Eficiente para parámetros pequeños	Una dimensión estructural es reducida.

Caso conductor: Knapsack 0/1

Dados objetos con peso w_i y beneficio v_i , seleccionar un subconjunto que maximice:

$$\text{máx} \sum_{i=1}^n v_i x_i \quad \text{sujeto a} \quad \sum_{i=1}^n w_i x_i \leq W, \quad x_i \in \{0, 1\}.$$

Enfoques exactos

- enumeración: $\mathcal{O}(2^n)$;
- Programación Dinámica: $\mathcal{O}(nW)$;
- Ramificar-Podar con cotas.

Enfoques no exactos

- heurísticas greedy;
- escalamiento de valores;
- FPTAS con error controlado.

Actividad: seleccionar una estrategia

Para cada escenario, proponga una estrategia y justifique la elección.

- ① Se requiere el óptimo para 25 objetos y existe una buena cota.
- ② Hay 10 000 objetos y se admite un error relativo pequeño.
- ③ La capacidad W es 2 000 y el número de objetos es moderado.
- ④ Se necesita responder en milisegundos y no existe garantía formal conocida.

Criterios de justificación

Tamaño de entrada, estructura, necesidad de optimalidad, garantía, memoria y tiempo disponible.

Teoría versus práctica

Dos algoritmos con la misma complejidad asintótica pueden diferir por:

- constantes ocultas;
- distribución y estructura de las entradas;
- localidad de memoria y uso de caché;
- asignaciones dinámicas y sobrecarga de funciones;
- paralelismo, compilador y lenguaje;
- calidad de la implementación.

Regla del curso

La evidencia experimental complementa el análisis, pero no lo sustituye. Un gráfico aislado no demuestra una complejidad asintótica.

Preparación para el laboratorio 1

Instrumentaremos algoritmos con tiempo y conteo de operaciones.

```
from time import perf_counter

def medir(algoritmo, entrada):
    inicio = perf_counter()
    resultado, operaciones = algoritmo(entrada)
    tiempo = perf_counter() - inicio
    return resultado, operaciones, tiempo
```

- Repetir las mediciones y usar una medida representativa.
- Separar generación de datos y ejecución.
- Conservar instancias para reproducir resultados.
- Validar la corrección antes de comparar tiempos.

Cierre: cinco ideas esenciales

- 1 La complejidad depende del tamaño de la representación.
- 2 P reúne problemas de decisión resolubles en tiempo polinomial.
- 3 NP reúne problemas afirmativos con certificados verificables en tiempo polinomial.
- 4 NP-hard y NP-completo describen dificultad mediante reducciones.
- 5 La estrategia depende de optimalidad, garantía, estructura, parámetros y recursos.

Siguiente sesión

Algoritmos de aproximación aplicados a scheduling en máquinas idénticas.

Responda individualmente:

- 1 ¿Por qué $\mathcal{O}(nW)$ puede ser pseudo-polinomial?
- 2 ¿Cuál es la diferencia entre NP-hard y NP-completo?
- 3 ¿En qué dirección debe plantearse una reducción para demostrar que un problema nuevo es NP-hard?
- 4 ¿Cuándo convendría aproximación en lugar de Ramificar-Podar?

Referencias y recursos

-  T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.^a ed. MIT Press, 2022.
-  J. Kleinberg y É. Tardos. *Algorithm Design*. Pearson/Addison-Wesley, 2005.
-  S. Arora y B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.
-  MIT OpenCourseWare. *Advanced Algorithms*.
<https://ocw.mit.edu/courses/6-854j-advanced-algorithms-fall-2005/>
-  CP-Algorithms. *Algorithms for Competitive Programming*. <https://cp-algorithms.com/>